

Progetto EGO-CH — Deep Learning

Relazione Finale

Nome Cognome

Nome Cognome

Nome Cognome

Anno Accademico 2025/2026

Indice

1	Introduzione	2
2	Il Dataset EGO-CH	3
3	Task 2 — Point of Interest Recognition	4
3.1	Definizione del Problema	4
3.2	La Baseline del Paper Originale	4
3.2.1	Architettura e Protocollo	4
3.2.2	Risultati del Paper (Tabella 4)	5
3.2.3	Analisi delle Difficoltà Evidenziate	5
3.3	La Nostra Proposta: YOLOv8	5
3.3.1	Motivazione della Scelta Architetturale	5
3.3.2	Approccio End-to-End	6
3.4	Pipeline Implementativa	6
3.4.1	Fase 1: Preprocessing e Riformattazione del Dataset	6
3.4.2	Fase 2: Training	8
3.5	Esperimenti e Risultati	9
3.5.1	Setup Sperimentale	9
3.5.2	Risultati Quantitativi	9
3.5.3	Curve di Addestramento	10
3.5.4	Analisi Qualitativa	11
3.5.5	Matrice di Confusione	13
3.6	Discussione	13
3.6.1	Confronto con la Baseline del Paper	13
3.6.2	Nota sulla Comparabilità delle Metriche	14
3.6.3	Trade-off Efficienza–Accuratezza	14
4	Conclusioni	15

Capitolo 1

Introduzione

Il progetto si colloca nell’ambito della *Egocentric Vision* applicata alla fruizione del patrimonio culturale. Il dataset **EGO-CH** [1], proposto da Ragusa et al., raccoglie video in prima persona registrati da visitatori di due siti culturali siciliani — Palazzo Bellomo (Siracusa) e il Monastero dei Benedettini (Catania) — e definisce tre task fondamentali per la comprensione automatica del comportamento del visitatore:

1. **Task 1 — Room-based Localization:** classificazione frame-per-frame dell’ambiente in cui si trova il visitatore.
2. **Task 2 — Point of Interest Recognition:** rilevamento e classificazione dei Punti di Interesse (POI) nei frame egocentrici tramite object detection.
3. **Task 3 — Object Retrieval:** recupero dell’immagine di riferimento di un oggetto a partire da una query egocentrica, mediante matching nello spazio latente.

La presente relazione descrive nel dettaglio il lavoro svolto sul **Task 2**, che ha costituito il contributo personale dell’autore al progetto di gruppo.

Capitolo 2

Il Dataset EGO-CH

Il dataset EGO-CH è composto da video egocentrici acquisiti mediante dispositivi indossabili (HoloLens, GoPro) all'interno di due siti culturali:

- **Palazzo Bellomo** (Siracusa): 22 ambienti, 191 Points of Interest, video di training (57 clip, media 1.4 min) e test (10 clip, media 31.3 min) a risoluzione 1280×720 @ 29.97 fps.
- **Monastero dei Benedettini** (Catania): 4 ambienti, 35 POI (inclusi elementi architettonici), video di training (48 clip) e test (455 clip brevi + 60 visite reali) a risoluzione 1216×684 / 1408×792 .

Per ciascun POI il dataset fornisce annotazioni sotto forma di bounding box in formato testuale, associate frame per frame ai video di riferimento. La natura dei contenuti egocentrici comporta sfide significative quali motion blur, occlusioni parziali, variazioni estreme di illuminazione e ambiguità semantica tra POI visivamente simili.

Capitolo 3

Task 2 — Point of Interest Recognition

3.1 Definizione del Problema

Il Task 2, denominato *Point of Interest Recognition*, consiste nel rilevare e classificare i Punti di Interesse (POI) all'interno di frame estratti da video egocentrici. Si tratta di un problema di **object detection**: per ogni frame il modello deve predire un insieme di bounding box, ciascuna associata alla classe del POI corrispondente, nella forma $(x_{\text{center}}, y_{\text{center}}, w, h, \text{class})$.

A differenza dei task 1 e 3, che operano rispettivamente a livello di classificazione globale del frame e di matching tra embedding, il Task 2 richiede una localizzazione spaziale fine-grained degli oggetti di interesse, rendendo necessario l'impiego di architetture specifiche per la detection.

3.2 La Baseline del Paper Originale

3.2.1 Architettura e Protocollo

Gli autori del dataset EGO-CH hanno adottato **YOLOv3** [2] (Redmon e Farhadi, 2018) come baseline per il Task 2, motivando la scelta con le performance in tempo reale e la popolarità dell'architettura nel dominio del patrimonio culturale [1].

YOLOv3 è un detector *single-stage* basato su *anchor boxes*, con un backbone **Darknet-53** da circa 61 milioni di parametri. Le caratteristiche salienti della configurazione adottata nel paper sono:

- **Anchor boxes**: vengono utilizzate le anchor standard fornite dagli autori per il dataset COCO, senza ricalcolo specifico per il dominio egocentrico.
- **Soglia di detection**: ottimizzata per ciascun sito culturale sul validation set tramite grid search nel range $[5 \times 10^{-4}; 0.40]$:
 - Palazzo Bellomo: soglia = 0.01, learning rate = 0.001
 - Monastero dei Benedettini: soglia = 0.001, learning rate = 0.01
- **Metrica di valutazione**: **mean Average Precision (mAP)** con soglia IoU pari a 0.5 (equivalente a mAP@50), mediata su tutti i video di test.

3.2.2 Risultati del Paper (Tabella 4)

I risultati riportati nella Tabella 4 del paper originale sono:

Tabella 3.1: Risultati della baseline YOLOv3 dal paper EGO-CH [1] (Tabella 4).

Sito Culturale	mAP@50
Palazzo Bellomo (191 POI)	10.59%
Monastero dei Benedettini (35 POI)	15.45%

3.2.3 Analisi delle Difficoltà Evidenziate

Gli autori stessi riconoscono che i punteggi sono «in general very low», e identificano le seguenti cause principali:

1. **Natura atipica dei POI:** i punti di interesse non sono solo oggetti ben definiti (quadri, sculture), ma includono **elementi architettonici** come pavimenti, corridoi e scale — intrinsecamente difficili da delimitare con bounding box.
2. **Riconoscimento a livello di istanza:** a differenza dei benchmark classici di object detection (COCO, VOC), dove le classi sono categorie semantiche ampie (es. “cane”, “auto”), qui il modello deve discriminare tra *istanze* diverse dello stesso tipo di oggetto (es. quadri diversi in stanze diverse).
3. **Motion blur estremo:** i movimenti rapidi della testa del visitatore producono artefatti di sfocatura che degradano significativamente le feature estratte dal backbone.
4. **Variazioni di illuminazione:** il dataset del Monastero è stato raccolto nell’arco di 3 mesi, con condizioni di luce radicalmente diverse.
5. **Disparità tra siti:** Palazzo Bellomo, con 191 POI, risulta drasticamente più difficile del Monastero (35 POI), come confermato dal gap di 5 punti percentuali nella mAP.

3.3 La Nostra Proposta: YOLOv8

3.3.1 Motivazione della Scelta Architettuale

Per affrontare questo task abbiamo scelto di utilizzare **YOLOv8** [3] (Ultralytics, 2023) in sostituzione del più datato YOLOv3 (2018) impiegato nel paper originale.

La famiglia di modelli YOLO (*You Only Look Once*) è celebre nell’ambito della computer vision per la sua capacità di individuare e classificare gli oggetti presenti in un’immagine in un unico passaggio, garantendo un’elevata velocità di esecuzione. Le motivazioni principali che ci hanno spinto verso la versione 8 sono le seguenti:

1. **Maggiore accuratezza ed efficienza:** YOLOv8 rappresenta lo stato dell’arte moderno della rete. A parità di risorse computazionali, riesce a estrarre caratteristiche visive molto più ricche e dettagliate rispetto a YOLOv3.

2. **Rilevamento senza griglie predefinite (Anchor-Free):** i modelli tradizionali come YOLOv3 dovevano basarsi su forme di rettangoli predefinite (chiamate *anchor box*) per tentare di indovinare dove si trovassero gli oggetti. YOLOv8 elimina questo vincolo rigido e impara a ritagliare direttamente i bordi degli oggetti, risultando molto più preciso quando si tratta di individuare elementi architettonici o forme irregolari.
3. **Capacità di distinguere oggetti simili:** grazie all'architettura riprogettata, la rete è in grado di distinguere meglio i dettagli fini tra oggetti semanticamente o visivamente molto simili (come quadri differenti o dettagli murali).
4. **Scalabilità dei modelli:** YOLOv8 offre diverse dimensioni di rete (da *Nano* fino a *X-Large*). Questo ci ha permesso di condurre un confronto diretto tra un modello leggerissimo (adatto a dispositivi mobili) e uno ad altissima capacità per il massimo della precisione.

3.3.2 Approccio End-to-End

A differenza dei Task 1 e 3, in cui le feature visive vengono estratte offline tramite un foundation model (DINOv2) e poi elaborate da classificatori o metriche di similarità a valle, nel Task 2 si adotta un approccio completamente **end-to-end**: il modello YOLOv8 opera direttamente sulle immagini raw ad alta risoluzione, estraendo internamente le feature tramite il proprio backbone convoluzionale e producendo le predizioni di detection in un unico passaggio forward.

Questa scelta è motivata dal fatto che i detector moderni come YOLOv8 sono progettati per apprendere rappresentazioni spazialmente localizzate, ottimizzate congiuntamente per la regressione delle bounding box e la classificazione — una proprietà che si perderebbe con un'estrazione di feature generiche pre-calcolate.

3.4 Pipeline Implementativa

3.4.1 Fase 1: Preprocessing e Riformattazione del Dataset

Il dataset EGO-CH presenta una **struttura eterogenea** tra i due siti culturali, che ha richiesto una fase di riformattazione non banale:

- **Palazzo Bellomo:** struttura piatta, con immagini e annotazioni (file `.txt` con bounding box in formato YOLO) raggruppate in cartelle separate ma con nomi corrispondenti.
- **Monastero dei Benedettini:** struttura gerarchica profonda, con immagini e annotazioni annidate per POI specifico (es. `Training/5_Antirefettorio/5.3_POI_nome/`).

Lo script `task2_format_yolo.py` è stato sviluppato per unificare queste strutture in un formato compatibile con YOLOv8:

1. **Scansione ricorsiva:** il dataset sorgente viene esplorato con `rglob` per individuare tutte le immagini (`.jpg`, `.jpeg`, `.png`) e i file di annotazione (`.txt`).

2. **Associazione immagine-label:** per ogni immagine viene cercato un file `.txt` con lo stesso *stem* (nome senza estensione). Solo le coppie valide vengono mantenute.
3. **Split randomizzato:** le coppie valide vengono mescolate con seed fissato (`seed = 42`) e suddivise in training (80%), validation (10%) e test (10%).
4. **Copia nella struttura YOLO:** le immagini e le label vengono copiate nelle rispettive sottocartelle:

```
yolo_dataset/  
+-- images/  
|   +-- train/  
|   +-- val/  
|   +-- test/  
+-- labels/  
|   +-- train/  
|   +-- val/  
|   +-- test/  
+-- dataset.yaml
```

5. **Generazione automatica del file `dataset.yaml`:** viene creato il file di configurazione richiesto da YOLOv8, contenente i percorsi delle cartelle, il numero di classi (dedotto dal class ID massimo nelle annotazioni) e i nomi delle classi.

La Figura 3.1 mostra la distribuzione delle istanze per classe e delle bounding box nel dataset formattato: si nota una marcata *long-tail distribution*, con poche classi dominanti e numerose classi con pochissime istanze.

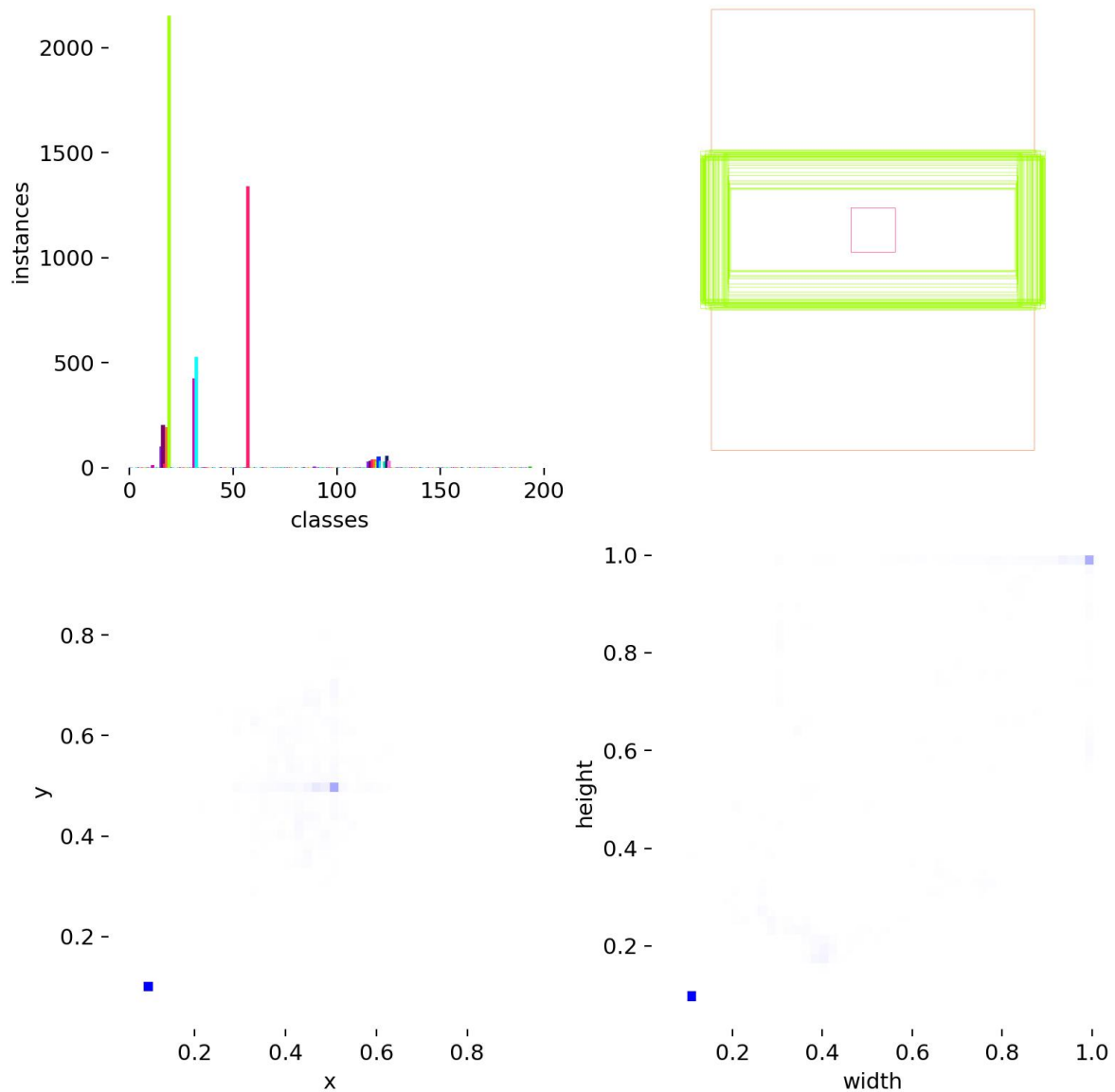


Figura 3.1: Distribuzione delle istanze per classe (in alto a sinistra), distribuzione spaziale dei centri delle bounding box (in basso a sinistra) e distribuzione delle dimensioni (larghezza vs. altezza, in basso a destra). Si nota la forte disomogeneità tra le classi e la predominanza di bounding box di grandi dimensioni, tipiche dei POI architettonici.

3.4.2 Fase 2: Training

L'addestramento è stato eseguito su un cluster HPC del Dipartimento di Matematica e Informatica (DMI) dell'Università di Catania, utilizzando GPU NVIDIA L40S tramite il sistema di job scheduling SLURM e il container runtime Apptainer.

Lo script `task2_train_yolo.py` sfrutta l'API della libreria `ultralytics` per configurare e lanciare l'addestramento caricando i pesi pre-addestrati sul dataset COCO.

La configurazione del job SLURM è stata definita nello script `run_task2_yolo.sh`, con le seguenti risorse allocate:

- 1 GPU NVIDIA L40S (22 GB VRAM allocati tramite `gres=shard`)

- 32 GB di RAM di sistema
- Timeout massimo di 12 ore

3.5 Esperimenti e Risultati

Sono stati condotti due esperimenti, variando la capacità del modello per analizzare il trade-off tra efficienza computazionale e accuratezza.

3.5.1 Setup Sperimentale

Tabella 3.2: Configurazione dei due esperimenti condotti.

Parametro	Run 1 (Nano)	Run 2 (X-Large)
Architettura	YOLOv8n	YOLOv8x
Pesi iniziali	yolov8n.pt (COCO)	yolov8x.pt (COCO)
Parametri	~3.3 M	~68.3 M
Epoche massime	100	300
Batch size	32	16
Risoluzione input	640×640	640×640
Optimizer	Auto (SGD)	Auto (SGD)
Learning rate (lr_0)	0.01	0.01
Data augmentation	Mosaic, RandAugment, Erasing	Mosaic, RandAugment, Erasing
Early stopping	Patience = 100	Patience = 100

3.5.2 Risultati Quantitativi

La Tabella 3.3 riporta il confronto completo tra la baseline del paper originale e i nostri due modelli sul sito del Monastero dei Benedettini.

Tabella 3.3: Confronto delle performance sul Monastero dei Benedettini. La metrica mAP@50 consente il confronto diretto con il paper originale, che utilizza $IoU = 0.5$. Vengono riportate anche le metriche COCO-style (mAP@[50:95]) per completezza.

Metrica	Baseline YOLOv3 [1]	YOLOv8n (Nano)	YOLOv8x (X-Large)
mAP@50	15.45%	19.92%	29.70%
mAP@[50:95]	—	14.82%	24.30%
Precision	—	0.489	0.685
Recall	—	0.218	0.298
Parametri	~61 M	~3.3 M	~68.3 M
Epoche effettive	—	100/100	269/300 (early stop)
Tempo di training	—	~1.6 ore	~7.2 ore

Tabella 3.4: Incrementi relativi rispetto alla baseline YOLOv3 del paper (mAP@50 = 15.45%).

Modello	mAP@50	Δ relativo vs. Baseline
Baseline YOLOv3 (Paper)	15.45%	—
YOLOv8n (Nano)	19.92%	+ 28.9%
YOLOv8x (X-Large)	29.70%	+ 92.2%

3.5.3 Curve di Addestramento

Le Figure 3.2 e 3.3 riportano l'andamento delle loss e delle metriche durante l'addestramento dei due modelli.

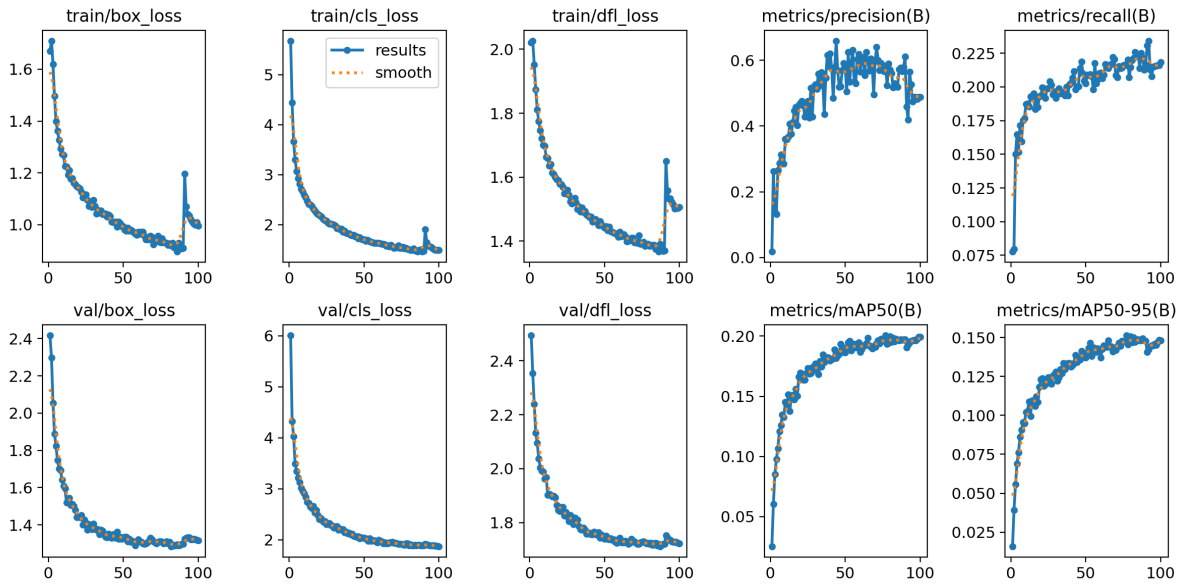


Figura 3.2: Curve di addestramento del modello YOLOv8n (Nano, 100 epoche). Le loss di training (box, cls, df) mostrano una convergenza regolare, mentre le metriche di validazione (mAP@50 e mAP@50-95) evidenziano un aumento costante fino all'ultima epoca, suggerendo che il modello avrebbe potuto beneficiare di un addestramento più lungo.

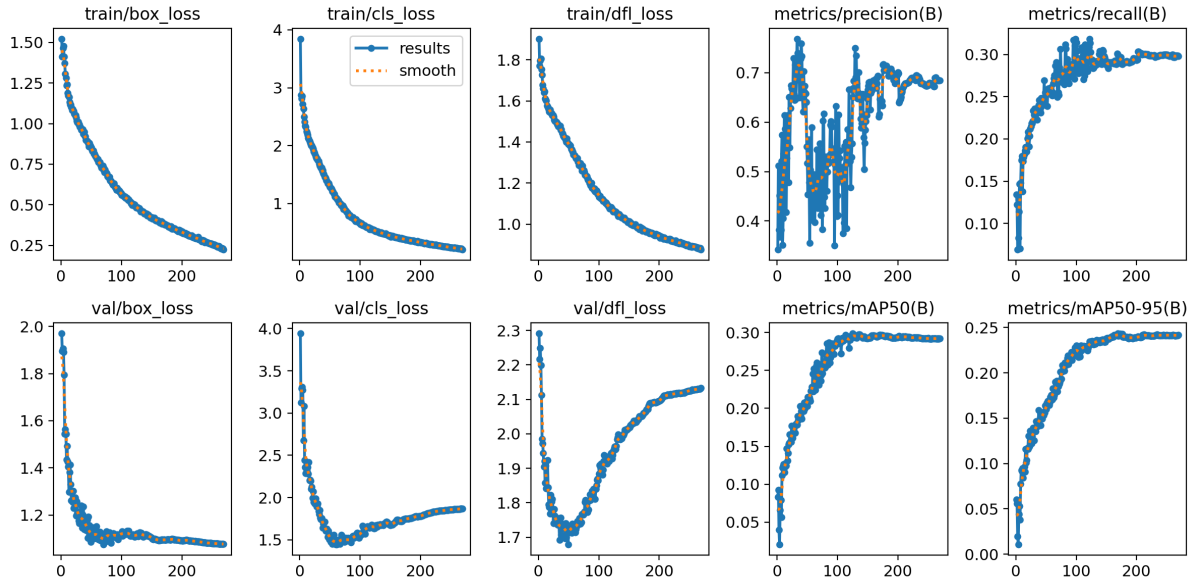


Figura 3.3: Curve di addestramento del modello YOLOv8x (X-Large, 269 epoche su 300 massime). Si osserva una convergenza più lenta ma più ricca: le loss di validazione raggiungono un plateau intorno all'epoca 100, mentre le metriche mAP continuano a crescere fino all'epoca ~ 170 , dopo la quale il meccanismo di early stopping (patience = 100) arresta l'addestramento all'epoca 269.

3.5.4 Analisi Qualitativa

La Figura 3.4 mostra un confronto tra le annotazioni ground truth e le predizioni del modello YOLOv8x su un batch di validazione. Si osserva che il modello è in grado di localizzare correttamente i POI anche in condizioni di illuminazione sfavorevole e su elementi architettonici complessi (archi, pavimenti, corridoi).



(a) Ground truth: bounding box annotate manualmente.



(b) Predizioni del modello YOLOv8x.

Figura 3.4: Confronto qualitativo tra ground truth (a) e predizioni YOLOv8x (b) su un batch di validazione. Le bounding box colorate indicano le classi dei POI. Si noti la varietà delle scene: archi, porte, pavimenti maiolicati, ambienti bui — tutti elementi tipici della sfida egocentrica in contesti culturali.

3.5.5 Matrice di Confusione

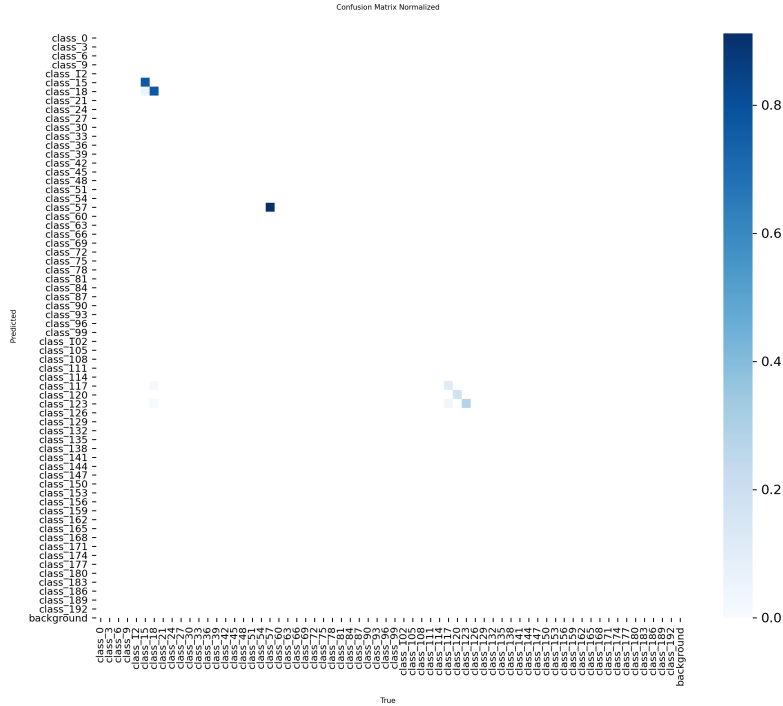


Figura 3.5: Matrice di confusione normalizzata del modello YOLOv8x. La forte sparsità della matrice riflette il numero elevatissimo di classi (>60) e la distribuzione long-tail del dataset: solo poche classi ben rappresentate (class_15, class_16, class_57) mostrano attivazioni significative sulla diagonale, mentre la maggior parte delle classi rare non viene mai predetta.

3.6 Discussione

3.6.1 Confronto con la Baseline del Paper

Il confronto con la baseline del paper originale rivela due risultati significativi:

YOLOv8n supera la baseline con $20\times$ meno parametri. Il modello Nano ha ottenuto un mAP@50 di **19.92%**, superiore del **+28.9%** rispetto al 15.45% di YOLOv3, pur disponendo di soli 3.3M di parametri contro i 61M del baseline. Questo risultato dimostra che le innovazioni architetturali di YOLOv8 (come l'approccio anchor-free e la struttura modernizzata) non solo compensano la riduzione drastica di capacità, ma portano a un netto miglioramento delle performance.

Il modello Nano risulta particolarmente adatto a scenari di deployment in tempo reale su dispositivi edge (es. visori HoloLens, smartphone), dove la potenza computazionale e il consumo energetico sono vincoli stringenti.

YOLOv8x raddoppia le performance della baseline. Il modello X-Large ha raggiunto un mAP@50 di **29.70%**, segnando un incremento relativo del **+92.2%** rispetto alla baseline. L'analisi dettagliata evidenzia:

- **Risoluzione dell’ambiguità visiva:** l’architettura anchor-free, combinata con il backbone CSPDarknet a piena capacità, ha permesso al modello di discriminare efficacemente tra POI visivamente simili.
- **Performance per classe non uniformi:** la rete X-Large ha raggiunto precisioni quasi perfette su classi ben rappresentate (es. Class_16: mAP 95.2%, Class_15: mAP 82.4%), mentre le classi con poche istanze o alta ambiguità visiva restano problematiche. Questa distribuzione long-tail è un limite intrinseco del dataset.
- **Early stopping funzionale:** l’addestramento si è arrestato automaticamente all’epoca 269 (su un massimo di 300), indicando che il modello ha raggiunto la convergenza intorno all’epoca 169 senza overfitting distruttivo.
- **Robustezza al motion blur:** la maggiore capacità di estrazione delle feature del backbone X-Large ha permesso di gestire il motion blur tipico dei video egocentrici in modo significativamente più robusto rispetto a YOLOv3.

3.6.2 Nota sulla Comparabilità delle Metriche

È importante sottolineare che il paper originale riporta la metrica **mAP@50** (IoU threshold = 0.5), che è la metrica standard per YOLOv3. I nostri risultati vengono riportati sia in formato mAP@50 (per il confronto diretto) sia nel formato COCO-style **mAP@[50:95]** (media su 10 soglie IoU da 0.5 a 0.95 con passo 0.05), che è lo standard moderno e produce valori sistematicamente inferiori in quanto più stringente. Il confronto diretto sulla metrica mAP@50 è il più corretto e mostra il miglioramento nella sua interezza.

3.6.3 Trade-off Efficienza–Accuratezza

Il confronto tra i due modelli mette in luce un chiaro trade-off:

- Se l’obiettivo è l’**accuratezza massima** per una valutazione accademica o per un sistema di analisi offline, **YOLOv8x** stabilisce il nuovo stato dell’arte con un margine netto.
- Se l’obiettivo prevede il **deployment su dispositivi indossabili** con risorse limitate, **YOLOv8n** offre un compromesso eccezionale, superando la precisione del baseline originale con una frazione dei parametri e un’inferenza in tempo reale.

Capitolo 4

Conclusioni

Il progetto ha affrontato i tre task fondamentali definiti dal benchmark EGO-CH, applicando tecniche moderne di Deep Learning per superare le baseline proposte dagli autori del dataset.

Per il **Task 1** (Room-based Localization), è stato impiegato un approccio basato su DINOv2 per l'estrazione delle feature e un modello Mamba (State Space Model) per la modellazione temporale delle sequenze video.

Per il **Task 2** (Point of Interest Recognition), oggetto del presente capitolo, la sostituzione di YOLOv3 con YOLOv8 ha prodotto un miglioramento fino al +92% in termini di mAP@50, dimostrando l'efficacia delle architetture anchor-free di ultima generazione anche su domini non convenzionali come la visione egocentrica in contesti culturali.

Per il **Task 3** (Object Retrieval), la sostituzione di VGG-19 con gli embedding DINOv2 ha portato a miglioramenti radicali della F1 Score, confermando la superiorità dei modelli fondazionali basati su Vision Transformer rispetto alle CNN tradizionali nel contesto del retrieval visuale.

I risultati complessivi confermano che l'adozione di architetture moderne e foundation model pre-addestrati consente di ottenere miglioramenti sostanziali rispetto alle baseline del 2020, anche senza modifiche al protocollo sperimentale o al dataset.

Bibliografia

- [1] F. Ragusa, A. Furnari, S. Battiato, G. Signorello, G. M. Farinella, *EGO-CH: Dataset and Fundamental Tasks for Visitors Behavioral Understanding using Egocentric Vision*, Pattern Recognition Letters, vol. 131, pp. 150–157, 2020.
- [2] J. Redmon, A. Farhadi, *YOLOv3: An Incremental Improvement*, arXiv preprint arXiv:1804.02767, 2018.
- [3] G. Jocher, A. Chaurasia, J. Qiu, *Ultralytics YOLOv8*, <https://github.com/ultralytics/ultralytics>, 2023.
- [4] M. Oquab et al., *DINOv2: Learning Robust Visual Features without Supervision*, Transactions on Machine Learning Research, 2024.